

System and software requirements

Before you download and build the Android kernel, ensure that your system meets the following requirements:

- Linux system (Linux running on a virtual machine will also work but is not recommended). Steps explained in this article are for Ubuntu 14.04 LTS to be specific. Other distributions should also work.
- Around 5 GB of free space to install the dependent software and build the kernel.
- Pre-built tool-chain.
- Dependent software should include GNU Make, libcurses5-dev, etc.
- Android kernel source (as mentioned earlier, this article describes the steps for the Samsung Galaxy Star kernel).
- Optionally, if you are planning to compile the whole Android platform (not just the kernel), a 64-bit system is required for Gingerbread (2.3.x) and newer versions.

It is assumed that the reader is familiar with Linux commands and the shell. Commands and file names are case sensitive. Bash shell is used to execute the commands in this article.

Step 1: Getting the source code

The Android Open Source Project (AOSP) maintains the complete Android software stack, which includes everything except for the Linux kernel. The Android Linux kernel is developed upstream and also by various handset manufacturers.

The kernel source can be obtained from:

- Google Android kernel sources: Visit <https://source.android.com/source/building-kernels.html> for details. The kernel for a select set of devices is available here.
- From the handset manufacturers or OEM website: I am listing a few links to the developer sites where you can find the kernel sources. Please understand that the links may change in the future.
 - Samsung: <http://opensource.samsung.com/>
 - HTC: <https://www.htcdev.com/>
 - Sony: Most of the kernel is available on [github](#).
- Developers: They provide a non-official kernel.

This article will use the second method—we will get the official Android kernel for Samsung Galaxy Star (GT-S5282). Go to the URL <http://opensource.samsung.com/> and search for GT-S5282. Download the file `GT-S5282_SEA_JB_Opensource.zip` (184 MB).

Let's assume that the file is downloaded in the `~/Downloads/kernel` directory.

Step 2: Extract the kernel source code

Let us create a directory 'android' to store all relevant files in the user's home directory. The kernel and Android NDK will be stored in the kernel and ndk directories,

respectively.

```
$ mkdir ~/android
$ mkdir ~/android/kernel
$ mkdir ~/android/ndk
```

Now extract the archive:

```
$ cd ~/Downloads/kernel

$ unzip GT-S5282_SEA_JB_Opensource.zip

$ tar -C ~/android/kernel -xzf Kernel.tar.gz
```

The unzip command will extract the zip archive, which contains the following files:

- `Kernel.tar.gz`: The kernel to be compiled.
- `Platform.tar.gz`: Android platform files.
- `README_Kernel.txt`: Readme for kernel compilation.
- `README_Platform.txt`: Readme for Android platform compilation.

If the unzip command is not installed, you can extract the files using any other file extraction tool.

By running the `tar` command, we are extracting the kernel source to `~/android/kernel`. While creating a sub-directory for extracting is recommended, let's avoid it here for the sake of simplicity.

Step 3: Install and set up the toolchain

There are several ways to install the toolchain. We will use the Android NDK to compile the kernel.

Please visit <https://developer.android.com/tools/sdk/ndk/index.html> to get details about NDK.

For 64-bit Linux, download Android NDK `android-ndk-r9-linux-x86_64-legacy-toolchains.tar.bz2` from http://dl.google.com/android/ndk/android-ndk-r9-linux-x86_64-legacy-toolchains.tar.bz2

Ensure that the file is saved in the `~/android/ndk` directory.



Note: To be specific, we need the GCC 4.4.3 version to compile the downloaded kernel. Using the latest version of Android NDK will yield to compilation errors.

Extract the NDK to `~/android/ndk`:

```
$ cd ~/android/ndk
# For 64 bit version
$ tar -jxf android-ndk-r9-linux-x86_64-legacy-toolchains.tar.bz2
```

Add the toolchain path to the PATH environment variable in `.bashrc` or the equivalent: